GRADE 11 · TERM 1

Arrays, Searching, Sorting & How Computers Work

A pencil-and-paper workbook: think it through first, learn the Delphi and the theory, then prove you can do it.

Name: _____

Class: _____ Date: _____

How to use this workbook

1. Work with a **pencil**. Mistakes are how we learn: rub out and try again.
2. Start each unit with **Think First**. Try it on your own before you read the explanation.
3. Read **Learn It**, then do the activities. Trace tables and code are written **by hand**.
4. At the end of each unit, tick the "I can" boxes you are sure of. Write down anything you still want to ask.
5. The **checkpoint test** is done under test conditions, with no notes.

In this workbook

1. A row of numbered boxes
2. Working with all the values
3. Finding things
4. Putting things in order
5. Dates and bits
6. Inside the machine
7. Software, networks and staying safe
8. Term checkpoint test

> **Matches the Annual Teaching Plan:** Term 1 — 1D arrays and nested loops; sum, average, minimum and maximum; linear and binary search; bubble and selection sort; parallel arrays; date methods; binary and decimal in code; motherboard, memory, storage, processing, operating systems, star networks, malware and backups.

UNIT 1 A row of numbered boxes

So far one variable held one value. A class of 30 learners needs 30 marks, and 30 separate variable names would be a nightmare. An **array** is one name for a whole row of numbered boxes.

By the end of this unit I can: (tick each box when you are sure)

- ☐ explain what an array is and why programmers use one
- ☐ use an index to read or change one box
- ☐ declare an array of a given size and data type
- ☐ fill an array with typed values, a loop or user input
- ☐ display every box with a FOR loop
- ☐ spot an index that is out of range

One thing I want to ask my teacher: _____

THINK FIRST · no computer needed

1.1 Row F in the school hall [4]

Row F of the school hall has **6 seats**, numbered 1 to 6 from the left. They are taken in order by Anele, Bongani, Chloe, Dumisani, Ethan and Fatima.

(a) Who sits in seat 4? (b) Bongani and Dumisani swap places. Who is in seat 2 and who is in seat 4 now?

(c) A latecomer is told: "Go to seat 9 in row F." What is wrong? (d) To send one learner to the right place *without using a name*, which **two** things must you say?

LEARN IT

An array is a row of numbered boxes

An **array** is a row of boxes that share **one name**. Every box holds one value, all of the **same data type**, and every box has a number called its **index**. In this course the first index is **1**. We write **arrPrices[3]** for "box 3 of arrPrices".

	1	2	3	4	5	6
arrPrices	12	8	15	6	20	9

Because the index can be a variable, a loop can visit every box: **arrPrices[i]** is box 1, then box 2, then box 3, as **i** counts. **Length(arrPrices)** tells you how many boxes there are (here 6). Box 0 and box 7 do **not exist**: using one is a **range-check error** and the program crashes.

1.2 Fill in the blanks [6]

An array stores many values of the same _____ under one _____. Each box has a number called the _____, and in this course the first index is ____. An array whose boxes are numbered 1 to 30 has ____ boxes. A _____ can visit every box because the index can be a variable. Using box 31 of that array causes a _____ error.

1.3 Read the boxes [8]

arrPrices is the array drawn in *Learn It* (boxes 1 to 6). Work out each value. In **arrPrices[i - 1]** the variable **i** holds 6.

Expression	Value	Expression	Value
arrPrices[3]	____	arrPrices[Length(arrPrices)]	____
arrPrices[1] + arrPrices[6]	____	arrPrices[i - 1]	____
arrPrices[2] * 3	____	arrPrices[1] + arrPrices[2] + arrPrices[3]	____
arrPrices[2 + 2]	____	arrPrices[7]	_____

LEARN IT

Declaring and filling an array

Declare the array in the **var** section: its name, the lowest and highest index, and the type. Then fill it box by box, with a loop, or with a constant list.

Delphi

```
var
  arrPrices : array[1..6] of Integer;     // 6 boxes, numbered 1 to 6
  arrNames  : array[1..30] of String;     // 30 boxes of text
  i : Integer;
begin
  arrPrices[1] := 12;                     // one box at a time
  arrNames[1]  := 'Thabo';
  for i := 2 to 6 do                      // user input in a loop
    arrPrices[i] := StrToInt(InputBox('Price', 'Price ' + IntToStr(i), ''));
  for i := 1 to 6 do                      // display every box
    memOut.Lines.Add(IntToStr(i) + ': R' + IntToStr(arrPrices[i]));
end;

const                                     // a list known in advance
  arrDays : array[1..7] of String = ('Mon','Tue','Wed','Thu','Fri','Sat','Sun');
```

1.4 Be the computer: filling with a loop [5]

Trace the code. Write what is stored in box **i** each time the loop body runs. (Box 1 is already filled.)

Delphi

```
arrCount[1] := 4;
for i := 2 to 6 do
  arrCount[i] := arrCount[i - 1] * 2 - 3;
```

i	arrCount[i - 1]	arrCount[i]
2	4	
3	5	
4	7	
5	11	
6	19	

1.5 Debug: the mark collector [3]

This code should ask for 5 marks and store them. Find **three** errors and write the corrected lines.

Delphi

```
var
  arrMark : array[1..5] of Integer;
  i : Integer;
begin
  for i := 1 to 6 do
    arrMark[i] := InputBox('Mark', 'Mark number ' + i, '');
end;
```

1.6 Write it: a week of sales **[7]**

A spaza shop records its sales (whole rand) for 7 days. Declare **arrSales** and **i**. Then write a loop that asks for each day's sales with an InputBox, stores it in the array, and (in a second loop) displays *Day 1: R...* to *Day 7: R...* in **memSales**.

UNIT 2 Working with all the values

Once the values are in an array, a loop can total them, average them and hunt for the biggest and smallest. You will also meet a loop *inside* a loop, which is how every box can be compared with every other box.

By the end of this unit I can: (tick each box when you are sure)

- ☐ total and average the values in an array
- ☐ find the highest and lowest value (and where it is)
- ☐ explain why the start value of a maximum matters
- ☐ trace a nested FOR loop
- ☐ decide how many times an inner loop runs

One thing I want to ask my teacher: _____

THINK FIRST · no computer needed

2.1 One taxi slip at a time [6]

A taxi driver has 5 fare slips in a row: R14, R9, R22, R17, R11. You may look at **one slip at a time**, in order, and you may only remember three things: the **total so far**, the **highest so far** and the **lowest so far**. Complete the table.

Slip	Fare (R)	Total so far	Highest so far	Lowest so far
1	14	____	____	____
2	9	____	____	____
3	22	____	____	____
4	17	____	____	____
5	11	____	____	____

Average fare = total ÷ number of slips = _____. Why did you start "highest so far" with the **first slip** rather than with 0? What goes wrong with a lowest-so-far that starts at 0?

THINK FIRST · no computer needed

2.2 Every pair shakes hands [3]

Aisha, Ben, Cebo and Dineo meet. **Every pair** shakes hands once. List the handshakes in an organised way: first Aisha with everyone after her, then Ben with everyone after him, and so on. How many handshakes are there? Predict how many there would be for 5 people and for 10 people.

LEARN IT

Totals, averages, highest and lowest

Use the same four ideas as the slips. A **running total** starts at 0. The **average** is the total divided by the number of boxes (use **/**, which gives a real number). **Highest so far** and **lowest so far** start as the **first box**, then the loop checks boxes 2 onwards. To know *where* the highest is, also keep its **position**.

Delphi

```
iTotal := 0;
for i := 1 to 5 do
  iTotal := iTotal + arrFare[i];
rAvg := iTotal / 5;

iMax := arrFare[1];   iMin := arrFare[1];   iPos := 1;
for i := 2 to 5 do
begin
  if arrFare[i] > iMax then
  begin
    iMax := arrFare[i];
    iPos := i;                  // remember WHERE it is
  end;
  if arrFare[i] < iMin then
    iMin := arrFare[i];
end;
lblAvg.Caption := FloatToStrF(rAvg, ffFixed, 8, 2);
```

2.3 Be the computer: highest and position [8]

arrMark holds 56, 71, 48, 90, 63, 90 (boxes 1 to 6). At the start iMax is 56 and iPos is 1. Complete a row for each pass.

Delphi

```
iMax := arrMark[1];
iPos := 1;
for i := 2 to 6 do
  if arrMark[i] > iMax then
  begin
    iMax := arrMark[i];
    iPos := i;
  end;
```

i	arrMark[i]	arrMark[i] > iMax ?	iMax	iPos
2	71			
3	48			
4	90			
5	63			
6	90			

2.4 Debug: the lowest mark [2]

This code should show the lowest of six marks in **lblLow**, but it always shows 0. Find **two** errors.

Delphi

```
iLow := 0;
for i := 1 to 6 do
  if arrMark[i] < iLow then
    iLow := arrMark[i];
lblLow.Caption := iLow;
```

LEARN IT

A loop inside a loop

A **nested loop** is a loop in the body of another loop. For **each** round of the outer loop, the inner loop runs **all** of its rounds. This is how you compare every box with the boxes after it. Look at where the inner loop starts: **i + 1**, so no box is paired with itself and no pair appears twice.

Delphi

```
// arrNames holds Aisha, Ben, Cebo, Dineo
for i := 1 to 3 do
  for j := i + 1 to 4 do
    memOut.Lines.Add(arrNames[i] + ' - ' + arrNames[j]);
```

2.5 Be the computer: nested loops [6]

Complete the table for the nested loop in *Learn It*: one row each time the inner loop body runs.

Delphi

```
for i := 1 to 3 do
  for j := i + 1 to 4 do
    memOut.Lines.Add(arrNames[i] + ' - ' + arrNames[j]);
```

i	j	Line added to memOut
1	2	
1	3	
1	4	
2	3	
2	4	
3	4	

2.6 Write it: weekly statistics [9]

arrSales holds the rand sales for 7 days (boxes 1 to 7, already filled). Write code that shows the **total** in **lblTotal**, the **average** to 2 decimal places in **lblAvg** and the **day number of the best day** in **lblBest**. Variables iTotal, iBest, i are declared.

2.7 Write it: any duplicates? **[7]**

arrCode holds 6 whole numbers (boxes 1 to 6). Use a **nested loop** and a Boolean flag **bDup** to decide whether any two boxes hold the same value. Then show *Duplicate found* or *All different* in **lblOut**.

UNIT 3 Finding things

Is Zinhle on the list? Where is the R20 ticket? Searching is one of the most common jobs a program does. You will learn two methods and decide which is better for which job.

By the end of this unit I can: (tick each box when you are sure)

- ☐ search an unsorted array with a loop and a flag
- ☐ explain why a binary search needs sorted data
- ☐ do a binary search by hand and count the guesses
- ☐ compare the number of steps for linear and binary search
- ☐ choose the right search for a problem

One thing I want to ask my teacher: _____

THINK FIRST · no computer needed

3.1 The raffle bag [3]

Ten raffle tickets lie in a bag in **no order**. You need to find ticket 6027, or find out that it is not there. You can only pull out one ticket at a time. (a) Describe your method. (b) What is the **fewest** tickets you might look at, and the **most**? (c) The tickets are now laid out on a table in order from smallest to biggest. How would you find 6027 more cleverly?

THINK FIRST · no computer needed

3.2 Higher or lower? [3]

I think of a whole number from 1 to 100. You guess and I answer **higher** or **lower**. Your best first guess is 50, because whatever I say, at most 50 numbers are left. Complete the table: after each guess, at most how many numbers are still possible? (Use the largest half each time and round down.)

After guess	1	2	3	4	5	6	7
Numbers possible	____	____	____	____	____	____	____

So the most guesses you can ever need for 100 numbers is ____.

LEARN IT

Linear search with a flag

A **linear search** looks at the boxes one by one from box 1. A **flag** (a Boolean variable) starts as False and is changed to True when the item is found. **Break** stops the loop at once, because there is no need to look further. It works on **any** array, sorted or not.

Delphi

```
bFound := False;
iPos := 0;
for i := 1 to 6 do
  if arrCode[i] = iTarget then
  begin
    bFound := True;
    iPos := i;
    Break;                      // stop looking
  end;
```

3.3 Be the computer: linear search [6]

arrCode holds 34, 18, 52, 7, 41, 29 (boxes 1 to 6) and **iTarget** is 7. Trace the search in *Learn It*: one row for each pass until the loop stops.

Delphi

```
// the loop from Learn It, with iTarget = 7
```

i	arrCode[i]	arrCode[i] = iTarget ?	bFound	iPos
1	34			
2	18			
3	52			
4	7			

3.4 Write it: find a learner [7]

arrNames holds **iCount** learner names in no particular order. The user types a name in **edtSearch**. Write code that shows *Found in box ...* (with the box number) or *Not on the list* in **lblOut**.

LEARN IT

Binary search (sorted arrays only)

Look at the middle box. If it is the target, you are done. If it is **too small**, the target can only be to its right, so move **iLow** up. If it is **too big**, move **iHigh** down. Every step throws away half of what is left. Stop when the target is found, or when **iLow > iHigh** (nothing left to look at). The array **must be sorted**, otherwise throwing away half is a guess.

Delphi

```
iLow := 1;
iHigh := 12;                    // number of boxes in use
bFound := False;
while (iLow <= iHigh) and (not bFound) do
begin
  iMid := (iLow + iHigh) div 2;
  if arrNum[iMid] = iTarget then
    bFound := True
  else if arrNum[iMid] < iTarget then
    iLow := iMid + 1            // target is to the right
  else
    iHigh := iMid - 1;          // target is to the left
end;
```

3.5 Which search? [5]

A linear search works on _____ array, but a binary search needs the array to be _____. For one million sorted values a binary search needs at most ____ guesses, while a linear search may need up to _____. A list of 8 prices typed in at random should be searched with a _____ search; a sorted list of 5 000 car registrations that is searched all day should use a _____ search.

3.6 Be the computer: binary search [8]

arrNum is sorted:

	1	2	3	4	5	6	7	8	9	10	11	12
arrNum	**5**	**8**	**12**	**15**	**19**	**23**	**27**	**31**	**38**	**44**	**50**	**57**

Trace a search for **iTarget = 44**. Write the missing values and say what happens to iLow or iHigh. The first row is done for you.

Delphi

```
// the binary search from Learn It: iLow := 1; iHigh := 12; iTarget = 44
```

Guess	iLow	iHigh	iMid	arrNum[iMid]	What happens?
1	1	12			
2	7	12			
3	10	12			
4	10	10			

3.7 Be the computer: not in the list [6]

Now search the same array for **iTarget = 20**, which is not there. The loop stops when iLow > iHigh. Write the final iLow and iHigh below the table.

Delphi

```
// same code as above, iTarget = 20
```

Guess	iLow	iHigh	iMid	arrNum[iMid]	What happens?
1	1	12			
2	1	5			
3	4	5			
4	5	5			
Stop	6	5			

3.8 Debug: the binary search [3]

This binary search hangs or misses values. Find **three** errors and write the corrections.

Delphi

```
iLow := 1;
iHigh := 12;
bFound := False;
while (iLow < iHigh) and (not bFound) do
begin
  iMid := (iLow + iHigh) / 2;
  if arrNum[iMid] = iTarget then
    bFound := True
  else if arrNum[iMid] < iTarget then
    iHigh := iMid - 1
  else
    iLow := iMid + 1;
end;
```

UNIT 4 Putting things in order

A sorted array can be searched fast, printed as a league table and checked at a glance. Computers sort by repeating very small steps: compare two values, maybe swap them. You will sort on paper first, exactly as the computer does.

By the end of this unit I can: (tick each box when you are sure)

- ☐ swap two values safely using a temporary variable
- ☐ trace a bubble sort pass by pass
- ☐ trace a selection sort pass by pass
- ☐ compare the two sorts
- ☐ keep parallel arrays together when sorting
- ☐ write the code for a sort

One thing I want to ask my teacher: _____

THINK FIRST · no computer needed

4.1 Swapping two drinks [3]

A glass of milk is in glass A and a glass of juice is in glass B. You must end with juice in A and milk in B, but you may only pour from one glass into another. (a) Can you do it with only these two glasses? (b) What do you need, and what are the steps? (c) Now write the same steps for two array boxes holding R12 and R30, using a spare variable **iTemp**.

THINK FIRST · no computer needed

4.2 Sort the queue by neighbours only [5]

Five learners wear numbers: **40, 15, 33, 8, 27**. They may only compare **two neighbours**: if the left one has the bigger number they swap places. Walk along the line from the left, comparing every pair. That is one **walk**. Write the line after each walk, and count the swaps. Stop after a walk with no swaps. (Use five scraps of paper if it helps.)

Walk	Line after the walk	Swaps
1	_____	____
2	_____	____
3	_____	____
4	_____	____

What do you notice about the **biggest** number after the first walk?

LEARN IT

Two ways to sort: bubble sort and selection sort

Bubble sort repeats the neighbour walk: in each pass it compares box **j** with box **j + 1** and swaps them if they are out of order. After pass 1 the biggest value is at the end, after pass 2 the second biggest is in place, and so on, so the inner loop can stop one box earlier each pass. The swap needs a spare variable, **iTemp**. For biggest first, change > to <.

Delphi

```
for i := iCount - 1 downto 1 do          // passes
  for j := 1 to i do                     // compare neighbours
    if arrNum[j] > arrNum[j + 1] then
    begin
      iTemp := arrNum[j];                // swap with a spare
      arrNum[j] := arrNum[j + 1];
      arrNum[j + 1] := iTemp;
    end;
```

Selection sort selects the smallest value of the whole array and swaps it into box 1, then the smallest of boxes 2 onwards into box 2, and so on. It remembers the *position* of the smallest so far in **iMin** and swaps only once per pass.

Delphi

```
for i := 1 to iCount - 1 do
begin
  iMin := i;                             // smallest so far is box i
  for j := i + 1 to iCount do
    if arrNum[j] < arrNum[iMin] then
      iMin := j;                         // a smaller one: remember WHERE
  if iMin <> i then
  begin
    iTemp := arrNum[i];
    arrNum[i] := arrNum[iMin];
    arrNum[iMin] := iTemp;
  end;
end;
```

4.3 Be the computer: bubble sort [8]

arrNum holds 27, 14, 31, 9, 22. Write the whole array after each pass, and the number of swaps in that pass.

Delphi

```
// the bubble sort from Learn It, with iCount = 5
```

Pass	Swaps	arrNum after the pass
1		
2		
3		
4		

4.4 Be the computer: selection sort [8]

Sort the same array, 27, 14, 31, 9, 22, with a selection sort. For each pass write the smallest value of the unsorted part, the box it was found in, and the array after the swap.

Delphi

```
// the selection sort from Learn It, with iCount = 5
```

Pass	Smallest in the unsorted part	Found in box	arrNum after the pass
1			
2			
3			
4			

4.5 Compare the two sorts [4]

Count the swaps in your two traces. Bubble sort made ____ swaps and selection sort made ____ swaps. The sort that moved values fewer times here was _____ sort, because it swaps only _____ per pass, after it has found the smallest value.

THINK FIRST · no computer needed

4.6 Marks without names [4]

Mr Dlamini keeps two lists with the **same box numbers**: names in one, marks in the other. He sorts only the marks, highest first, and leaves the names where they were. Write the sorted marks next to each name.

Box	Name (not moved)	Original mark	Marks after sorting only the marks
1	Lebo	62	____
2	Kyle	85	____
3	Ayanda	47	____
4	Zinhle	91	____
5	Pieter	70	____

Who does the list now say scored 91? Who *really* scored 91? What should the program do whenever it swaps two marks?

LEARN IT

Parallel arrays

Parallel arrays have boxes with the same index that belong together: **arrNames[3]** and **arrMarks[3]** are the same learner. To sort by marks, compare the marks but swap **both** arrays, each with its own spare variable.

Delphi

```
if arrMarks[j] > arrMarks[j + 1] then
begin
  iTemp := arrMarks[j];                // swap the marks ...
  arrMarks[j] := arrMarks[j + 1];
  arrMarks[j + 1] := iTemp;
  sTemp := arrNames[j];                // ... and the names, at the SAME boxes
  arrNames[j] := arrNames[j + 1];
  arrNames[j + 1] := sTemp;
end;
```

4.7 Be the computer: parallel sort [6]

Selection sort, smallest mark first. **arrNames** is Thabo, Amy, Sipho, Nandi and **arrMarks** is 58, 73, 41, 66. Write both arrays after each pass.

Delphi

```
// selection sort on arrMarks, swapping arrNames in step
```

Pass	arrMarks after the pass	arrNames after the pass
1		
2		
3		

4.8 Write it: sort a league table **[8]**

arrNames and **arrScores** are parallel arrays with **iCount** boxes in use. Write a bubble sort that puts the **highest score first** and keeps every name with its score. (iTemp, sTemp, i and j are declared.)

UNIT 5 Dates and bits

Delphi can read the clock and do calendar sums for you. You will also teach the computer to convert between binary and decimal, using the loops and string skills you already have.

By the end of this unit I can: (tick each box when you are sure)

- ☐ display the date and time with DateToStr and TimeToStr
- ☐ use YearOf, MonthOf, DayOf and IsLeapYear
- ☐ work out a person's exact age from two dates
- ☐ convert a binary string to decimal in code
- ☐ convert a decimal number to binary in code

One thing I want to ask my teacher: _____

THINK FIRST · no computer needed

5.1 Why does February sometimes have 29 days? [3]

The Earth needs about **365 and a quarter days** to go round the sun, but a calendar year has 365. (a) If nothing is done, by how many days is the calendar wrong after 4 years? (b) What fix do we use? (c) Which of these years are leap years under the simple rule "every year that divides by 4": **2023, 2024, 2026, 2028**?

LEARN IT

Date methods

Delphi stores a date and time in a **TDateTime** value. **Now** is the date and time on the computer clock. **YearOf**, **MonthOf**, **DayOf** and **IsLeapYear** need **DateUtils** in the uses clause. The text produced by DateToStr and TimeToStr depends on the computer's regional settings (in South Africa usually like 2026/03/15 and 14:32:07).

Method	Gives	Example
DateToStr(d)	the date as text	lblDate.Caption := DateToStr(Now);
TimeToStr(t)	the time as text	lblTime.Caption := TimeToStr(Now);
YearOf(d), **MonthOf(d)**, **DayOf(d)**	Integer year, month (1–12), day (1–31)	iYear := YearOf(dtpBirth.Date);
IsLeapYear(y)	Boolean for a year number	bLeap := IsLeapYear(2028);

Leap year rule: a year is a leap year if it divides by **400**, or divides by 4 but **not** by 100. So 2000 and 2024 are leap years; 1900 and 2100 are not.

Delphi

```
bLeap := (iYear mod 400 = 0) or ((iYear mod 4 = 0) and (iYear mod 100 <> 0));
```

5.2 Leap or not? [8]

Use the leap year rule and write *Yes* or *No*.

1900: ____ 2000: ____ 2023: ____ 2024: ____
2026: ____ 2028: ____ 2100: ____ 2400: ____

5.3 Be the computer: exact age [8]

Subtracting the two years only gives the age a person *will be* this year. If the birthday has not happened yet, subtract one. It is still to come when today's month is before the birth month, or the months are equal and today's day is before the birth day. Today is **15 March 2026**. Work out each learner's exact age.

Delphi

```
iAge := YearOf(dtToday) - YearOf(dtBorn);
if (MonthOf(dtToday) < MonthOf(dtBorn)) or
   ((MonthOf(dtToday) = MonthOf(dtBorn)) and (DayOf(dtToday) < DayOf(dtBorn))) then
  Dec(iAge);                // birthday still to come this year
```

Born (yyyy/mm/dd)	Year difference	Birthday still to come?	Exact age
2009/03/20			
2010/03/15			
2008/12/01			
2011/01/30			

THINK FIRST · no computer needed

5.4 The doubling trick [3]

Read a binary number from the **left**. Start with a running total of 0. For each digit: **double** the total, then **add the digit**. Do it for 1101.

Digit	Working	Total
1	_____	____
1	_____	____
0	_____	____
1	_____	____

Check with place values (8, 4, 2, 1): 1101 = 8 + 4 + 0 + 1 = ____. Now use the trick on 10010: ____.

LEARN IT

Binary and decimal in code

The doubling trick is a loop over the characters of the string. **Ord(c) - Ord('0')** turns the character '1' into the number 1. To go the other way, repeat: the remainder after dividing by 2 (**mod 2**) is the next binary digit from the **right**, so each new digit is put at the **front** of the string. Divide the number by 2 (**div 2**) and repeat until it is 0.

Delphi

```
// binary string -> decimal
iDec := 0;
for i := 1 to Length(sBin) do
  iDec := iDec * 2 + (Ord(sBin[i]) - Ord('0'));

// decimal -> binary string
sBin := '';
while iNum > 0 do
begin
  sBin := IntToStr(iNum mod 2) + sBin;   // new digit goes in FRONT
  iNum := iNum div 2;
end;
```

5.5 Be the computer: binary to decimal [8]

sBin is '10110101'. Complete a row for each pass of the loop.

Delphi

```
iDec := 0;
for i := 1 to Length(sBin) do
  iDec := iDec * 2 + (Ord(sBin[i]) - Ord('0'));
```

i	sBin[i]	iDec after the pass
1	1	
2	0	
3	1	
4	1	
5	0	
6	1	
7	0	
8	1	

5.6 Be the computer: decimal to binary [8]

iNum is 37. Complete a row for each pass of the loop.

Delphi

```
sBin := '';
while iNum > 0 do
begin
  sBin := IntToStr(iNum mod 2) + sBin;
  iNum := iNum div 2;
end;
```

Pass	iNum before	iNum mod 2	sBin after	iNum after
1	37			
2	18			
3	9			
4	4			
5	2			
6	1			

5.7 Write it: check, then convert **[8]**

The user types a binary number in **edtBin**. Write code that first checks that **every character** is a 0 or a 1 (use a Boolean flag). If it is valid, show its decimal value in **lblDec**; otherwise show *Not a binary number*.

UNIT 6 Inside the machine

Programs run on hardware. In this unit you will draw and label the motherboard, follow what happens when the computer starts, and decide which kind of memory and storage suits a job. Write your theory answers in **sentences**.

By the end of this unit I can: (tick each box when you are sure)

- ☐ label the main parts of a motherboard and say what each does
- ☐ name the three buses and what they carry
- ☐ put the start-up steps (BIOS/UEFI, POST) in order
- ☐ compare RAM and ROM, and cache levels L1, L2 and L3
- ☐ choose between an HDD and an SSD for a job and give reasons
- ☐ match a slow computer to the upgrade that helps

One thing I want to ask my teacher: _____

THINK FIRST · no computer needed

6.1 A town on a board [3]

A small town has: a **municipal office** where every decision is made, a **shop** next door holding only what is needed today, a huge **depot** out of town where everything is stored for years, a network of **roads** joining them, and a **power station**. Which computer part does each one match? What do the roads carry?

LEARN IT

The motherboard and its buses

The **motherboard** is the main circuit board: everything plugs into it or is built onto it. The **CPU** sits in its socket. **RAM** goes into DIMM slots. Extra cards (graphics, sound, network) go into **PCI Express (PCIe)** slots. Hard drives and SSDs connect to **SATA** ports. The **power connector** brings power from the power supply, a small **CMOS battery** keeps the clock running when the computer is off, and the **BIOS/UEFI chip** holds the start-up program. A **bus** is a set of wires carrying signals between parts: the **data bus** carries the data and instructions, the **address bus** carries the memory address where the data is found, and the **control bus** carries signals that coordinate the parts (for example "read" or "write"). A wider data bus moves more bits at once.

6.2 The three buses [3]

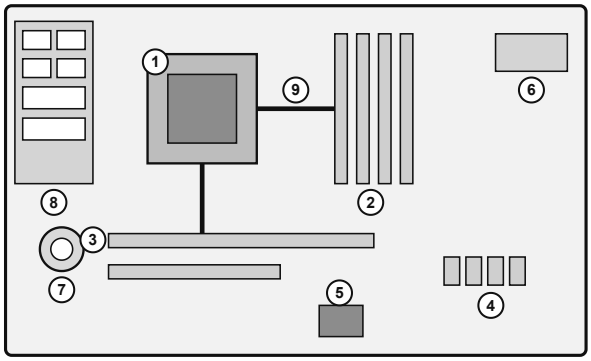
The _____ bus carries data and instructions between the parts. The _____ bus carries the location in memory that the CPU wants to read or write. The _____ bus carries signals such as "read" and "write" that coordinate the parts.

THINK FIRST · no computer needed

6.3 Where is your pencil? [2]

Rank these places by how **fast** you can get a pencil and by how many pencils they can **hold**: (A) in your hand, (B) in your pencil case on the desk, (C) in your school bag, (D) in your locker, (E) at home. What do you notice about speed and size? Computers use the same idea for their memory.

6.4 Label the motherboard [9]



Write the name of each numbered part. *Word bank: CPU socket · RAM (DIMM) slots · PCIe slot · SATA ports · BIOS/UEFI chip · power connector · CMOS battery · rear ports (USB, HDMI, network) · bus*

No.	Part	No.	Part	No.	Part
1	_____	4	_____	7	_____
2	_____	5	_____	8	_____
3	_____	6	_____	9	_____

LEARN IT

BIOS/UEFI and POST

The **BIOS** (Basic Input/Output System) or its modern replacement **UEFI** is a small program stored on a chip on the motherboard. It runs the moment you press the power button. First it runs the **POST** (Power-On Self-Test), which checks that the CPU, RAM, keyboard and drives respond; a fault produces beeps or an error screen. Then it finds the drive that holds the operating system and passes control to the **bootloader**, which loads the operating system into RAM. Think of a caretaker who switches on the power and checks the water, lights and alarm before the principal takes over: problems must be found *before* the day starts. Without a working BIOS/UEFI the computer cannot start, even if the hard drive is perfect.

6.5 Start-up order [6]

Number the steps from 1 to 6 in the order they happen.

____ The operating system is loaded into RAM and the login screen appears
____ POST checks the CPU, RAM, keyboard and drives
____ The power button is pressed and power flows to the motherboard
____ The bootloader is found and started
____ The BIOS/UEFI program starts running from its chip
____ The BIOS/UEFI looks for the drive that holds the operating system

LEARN IT

Memory and storage

Cache is a tiny, very fast memory on or beside the CPU that keeps copies of the data and instructions the CPU uses most. **L1** is the smallest and fastest (inside each core), **L2** is bigger and a little slower, and **L3** is the largest of the three and is shared by the cores. The CPU looks in L1, then L2, then L3, then RAM, then storage. Keeping a copy of often-used data in a faster place is called **caching**; a web browser's cache does it for web pages.

RAM is the computer's working memory: it holds the programs and files in use now, it is read-write and **volatile** (it loses everything when the power goes off). **ROM** is read-only memory that keeps its contents without power; it holds permanent start-up instructions such as the BIOS/UEFI. **Storage** keeps files when the computer is off. A **hard disk drive (HDD)** stores data magnetically on spinning platters read by a moving head: it is cheap per gigabyte and has a large capacity, but it is slower, noisy and can be damaged if dropped. A **solid state drive (SSD)** stores data in flash memory chips with no moving parts: it is much faster, silent, uses less power and survives knocks, but costs more per gigabyte.

6.6 RAM or ROM? [6]

Complete the comparison.

	RAM	ROM
Loses its contents when the power goes off?		
Can programs easily write new data to it?		
Typical contents		

6.7 Explain in a paragraph: RAM and ROM [4]

Compare RAM and ROM in a short paragraph of full sentences. Mention what each one keeps, and what happens to it when the power goes off.

6.8 Scenarios: HDD or SSD? [6]

Recommend an HDD or an SSD for each situation and explain your choice in two or three sentences.
(a) Thandi carries her laptop to school in a bag on a crowded taxi every day, and it must start quickly because lessons are only 50 minutes long.
(b) A school wants to archive ten years of scanned records (about 20 TB) on a small budget. The files are opened only a few times a year.

6.9 Which upgrade helps? [6]

A computer that slows down when many programs are open needs more _____. One that takes minutes to start from an old spinning drive needs an _____. Jerky, low-detail game graphics point to a better _____ (graphics card). A huge calculation that takes very long needs a faster _____ with more cores. If the drive light stays on because programs are constantly swapped to the drive, the computer is _____ and needs more _____.

UNIT 7 Software, networks and staying safe

Hardware does nothing without software, and a computer on its own is only half as useful as one on a network. You will compare operating systems and translators, see how a computer does many things at once, draw a star network and plan how to protect data.

By the end of this unit I can: (tick each box when you are sure)

- ☐ compare stand-alone, network, embedded and mobile operating systems
- ☐ tell multitasking, multithreading and multiprocessing apart
- ☐ explain virtual memory and thrashing
- ☐ compare a compiler with an interpreter
- ☐ draw a star network and say what a NIC, modem, switch and router do
- ☐ name common malware and plan backups

One thing I want to ask my teacher: _____

THINK FIRST · no computer needed

7.1 The tuck shop at break [2]

At break 200 learners queue at the tuck shop. Mrs Dlamini copes in three different ways. **(a)** She works alone, but swaps quickly between serving a customer, using the till and taking pies out of the warmer, so everything keeps moving. **(b)** One big order, a hamper for a staff meeting, is cut into smaller jobs (packing, adding up, writing the slip) that are all in progress together while she is the only worker. **(c)** She hires two helpers, so three people really serve customers at the same moment. In which way is work *truly* being done at the same time? Why do (a) and (b) only look as if it is?

LEARN IT

Operating systems and doing many things at once

An **operating system (OS)** manages the hardware and runs programs. A **stand-alone** OS serves one user on one computer (Windows 11 on a laptop). A **network** OS runs on a server and manages users, files, printers and security for many computers. An **embedded** OS is built into a device that does one job with very little memory (an ATM, a router, a smart TV). A **mobile** OS (Android, iOS) suits touchscreens and long battery life.

Multitasking: the OS gives each open program a tiny slice of time in turn, so they appear to run together. **Multithreading**: one program is split into **threads** that progress together, so a slow part such as a download does not freeze the rest. **Multiprocessing**: a computer with more than one CPU core really runs several tasks at the same instant.

LEARN IT

Virtual memory and translators

When RAM is full the OS uses part of the storage drive as extra memory, called **virtual memory**: parts of programs not needed right now are swapped out to the drive and swapped back later. It lets more programs run, but the drive is far slower than RAM. If the computer spends more time swapping than working, that is **thrashing**; the cure is to close programs or add **RAM**.

The CPU only understands **machine code**. A **compiler** translates the **whole** program before it runs and produces an executable file (Delphi does this): errors are reported first and the finished program runs fast. An **interpreter** translates and runs the program **one line at a time** (Python, JavaScript): it starts at once but runs slower, and a mistake only shows when that line is reached.

7.2 Which processing technique? [4]

Circle the best answer.

1	Music plays while you type, on a one-core computer.	**Multitasking**	**Multithreading**	**Multiprocessing**
2	A browser scrolls while a file downloads inside it.	**Multitasking**	**Multithreading**	**Multiprocessing**
3	A quad-core CPU runs four programs at one instant.	**Multitasking**	**Multithreading**	**Multiprocessing**
4	A phone gives each open app a time slice in turn.	**Multitasking**	**Multithreading**	**Multiprocessing**

7.3 Operating system types [4]

A laptop used by one person at home runs a _____ OS. A school server that controls log-ins and printers runs a _____ OS. The small system inside an ATM or a router is an _____ OS. Android on a touchscreen phone is a _____ OS.

7.4 Compiler or interpreter [3]

A _____ translates the whole program before it runs and produces an _____ file; Delphi uses one. An _____ translates and runs one line at a time, so a mistake on line 40 only shows when line 40 is reached.

7.5 Virtual memory [5]

When the RAM is full, the operating system uses space on the _____ as extra memory. This is called _____ memory. It is much _____ than real RAM. When the computer spends more time swapping data than doing useful work, we call this _____, and the best cure is to add more _____.

LEARN IT

A star network and its devices

In a **star topology** every device has its own cable to a central **switch**. If one cable or computer fails, the rest keep working, and it is easy to add a device or find a fault. But if the switch fails the whole network fails, and it needs more cable. Every device needs a **NIC** (network interface card) to join the network. The **switch** joins the devices of the LAN and sends each message only to the device it is meant for. The **router** joins the LAN to another network such as the internet and directs data using IP addresses. The **modem** converts between the digital signals of the computers and the signal on the fibre, phone or mobile line to the internet service provider.

7.6 Draw a star network [7]

Draw a star network for a small computer lab with **three learner PCs**, a **teacher PC** and a **printer**, connected to the **Internet**. Show a **switch** in the middle, a **router** and a **modem**. Label every device and write where the **NIC** is.

7.7 Network devices [4]

Circle the device described.

1	Joins one device to the network; every device has one	**NIC** **Modem** **Switch** **Router**
2	Sends data only to the device it is meant for	**NIC** **Modem** **Switch** **Router**
3	Joins the lab to the internet using IP addresses	**NIC** **Modem** **Switch** **Router**
4	Converts between digital signals and the line signal	**NIC** **Modem** **Switch** **Router**

LEARN IT

Malware and backups

Malware is software designed to cause harm. A **virus** attaches itself to a file and spreads when the file is opened. A **worm** spreads across a network by itself. A **trojan** pretends to be useful software but lets an attacker in. A **rootkit** hides deep in the OS so that an attacker can control the computer unseen. **Ransomware** encrypts your files and demands payment. **Spyware** secretly records what you do; **adware** shows unwanted adverts. Protection: antivirus, a firewall, updates, strong passwords and care with links and attachments.

A **backup** is a copy of data kept so that it can be restored. A **full** backup copies everything (quickest to restore). An **incremental** backup copies only what changed since the last backup of any kind (quickest to make, slowest to restore). A **differential** backup copies what changed since the last full backup. Copies can be kept **locally** (fast, but lost in a fire or theft), **off-site** or in the **cloud**. The **3-2-1 rule**: 3 copies, on 2 kinds of media, with 1 off-site.

7.8 Name the malware [6]

Word bank: *virus* · *worm* · *trojan* · *rootkit* · *ransomware* · *spyware*

Description	Malware
Attaches to a file and spreads when that file is opened	
Spreads across a network by itself without anyone opening a file	
A free game that secretly lets an attacker into your computer	
Hides deep in the operating system so that an attacker can control the PC unseen	
Locks your files and demands money to unlock them	
Secretly records the sites you visit and what you type	

7.9 Plan a backup [5]

The school secretary keeps all learner records on one PC. A power surge destroys its hard drive and the records are gone. In a short paragraph explain what backup plan would have saved the records. Mention how often, which type, and where the copies should be kept.

TERM CHECKPOINT

Arrays, algorithms and how computers work

Time: 90 minutes **Total:** 62 marks **Mark:** _____ / 62

No notes, no computer. Show all your working. Write Delphi code neatly and use correct indentation.

SECTION A — Trace and predict [12]

A1 [5]

arrFare holds 18, 25, 12, 30, 22 (boxes 1 to 5). Complete the table for each pass of the loop.

```delphi
iTotal := 0;
iBig := 0;
for i := 1 to 5 do
begin
  iTotal := iTotal + arrFare[i];
  if arrFare[i] > 20 then
    Inc(iBig);
end;
```

i	arrFare[i]	iTotal	iBig
1	18		
2	25		
3	12		
4	30		
5	22		

A2 [4]

arrNum holds 16, 7, 12, 4, 9. Write the number of swaps in each pass and the whole array after each pass.

```delphi
for i := 4 downto 1 do
  for j := 1 to i do
    if arrNum[j] > arrNum[j + 1] then
      swap arrNum[j] and arrNum[j + 1];
```

Pass	Swaps	arrNum after the pass
1		
2		
3		
4		

A3 **[3]**

arrCode is sorted: 3, 9, 14, 20, 26, 31, 38, 45, 52 (boxes 1 to 9). **iTarget** is 10. Write the value of iMid for each pass, in order, and the final value of bFound.

Delphi

```
iLow := 1;
iHigh := 9;
bFound := False;
while (iLow <= iHigh) and (not bFound) do
begin
  iMid := (iLow + iHigh) div 2;
  if arrCode[iMid] = iTarget then
    bFound := True
  else if arrCode[iMid] < iTarget then
    iLow := iMid + 1
  else
    iHigh := iMid - 1;
end;
```

SECTION B — Bits and dates [11]

B1 **[5]**

sBin is '11010'. Complete a row for each pass, then write the final value of iDec.

Delphi

```
iDec := 0;
for i := 1 to Length(sBin) do
  iDec := iDec * 2 + (Ord(sBin[i]) - Ord('0'));
```

i	sBin[i]	iDec after the pass
1	1	
2	1	
3	0	
4	1	
5	0	

B2 **[3]**

iNum is 11. Write the value of **sBin** after each pass of the loop, and its final value.

Delphi

```
sBin := '';
while iNum > 0 do
begin
  sBin := IntToStr(iNum mod 2) + sBin;
  iNum := iNum div 2;
end;
```

B3 **[3]**

dtpDate.Date is 29 February 2024. Write the value of each expression.

```Delphi
YearOf(dtpDate.Date)
MonthOf(dtpDate.Date)
IsLeapYear(YearOf(dtpDate.Date))
```

SECTION C — Write the code [24]

C1 **[8]**

arrPrice holds the prices (whole rand) of **iCount** tuck shop items. Write code that shows the **total** in **lblTotal**, the **average** to 2 decimal places in **lblAvg** and the **highest price** in **lblMax**.

C2 **[6]**

arrCode holds **iCount** whole numbers in no particular order. The user types a code in **edtCode**. Show the position of its **first** occurrence, or *Code not found*, in **lblOut**.

C3 **[7]**

Write a **selection sort** that arranges **arrNum[1..iCount]** from smallest to biggest. Variables i, j, iMin and iTemp are declared.

C4 **[3]**

This code should add up the marks in **arrMark[1..5]** that are 50 or more, and show how many there are. Find and correct **three** errors.

```Delphi
var
  arrMark : array[1..5] of Integer;
  i, iPass : Integer;
begin
  for i := 0 to 5 do
    arrMark[i] := StrToInt(InputBox('Mark', 'Enter a mark', ''));
  for i := 1 to 5 do
    if arrMark[i] >= 50 then
      iPass := iPass + 1;
  lblPass.Caption := iPass;
end;
```

SECTION D — Computers and networks [15]

D1 **[4]**

Word bank: *RAM* · *SSD* · *BIOS/UEFI* · *cache*. Write the term that fits each description.

Description	Term
Tiny, very fast memory on the processor that keeps copies of the data it uses most	
Flash storage with no moving parts	
Working memory that is wiped when the power goes off	
A program on a motherboard chip that runs the POST and starts the boot process	

D2 **[4]**

A computer with only 2 GB of RAM becomes very slow when many programs are open. In a short paragraph explain what the operating system does when the RAM is full, why this slows the computer down, and one way to fix it.

D3 **[3]**

Circle the best answer.

1	A browser keeps scrolling while a file downloads inside the same program.	**Multitasking**	**Multithreading**	**Multiprocessing**
2	A quad-core CPU runs four programs at the same instant.	**Multitasking**	**Multithreading**	**Multiprocessing**
3	The OS gives each open program a short slice of time in turn.	**Multitasking**	**Multithreading**	**Multiprocessing**

D4 **[4]**

Give one **advantage** and one **disadvantage** of a star network. Write each in a full sentence with a reason.

COMING UP NEXT TERM · warm-up

Notebooks and recipe cards

In Term 2 your programs will **remember** things by writing them in a file, and you will learn to break a big program into small, named helpers. Think about both without a computer.

(a) The stock notebook. A spaza owner writes one item and its quantity on each line of a notebook. At the end of the month she must read *every* line, add up the quantities, and then add three new items at the end without losing the old ones. Write the steps. How does she know she has reached the end? What would happen if the notebook is missing?

(b) The recipe cards. A bobotie recipe says: "Make the filling (card A). Make the topping (card B). Bake for 40 minutes." Card A says what to do with *onions and mince*. Why is the main recipe easier to read than one very long list? Card B needs to know *how many* people you are cooking for. How could the main recipe pass that number to the card?